

Лекция 3. Процедурная семантика, вычислимость, блок-схемы, структурное программирование

Курс: Информационные системы поддержки принятия решений

Источники: фото конспекта (3 фрагмента): процедурная семантика и применимость программы, теорема о выразимости функций на Милане, блок-схема алгоритма Евклида, список «Структурное программирование».

[дополнение] — добавлено мной; [неясно] — см. конец файла.

Ключевые определения и теоремы

Понятие	Суть
Программа с параметрами	$\Pi(x_1, \dots, x_m)$ — программа на Милане, у которой m входных переменных.
Запуск на входе	$x_i \leftarrow n_i$: входным переменным присваиваются числа $n_1, \dots, n_m$.
Протокол применения [дополнение: формулировка]	Последовательность состояний (значений переменных) по шагам выполнения — как таблица трассировки из лекции 2.
$\downarrow \Pi(n_1, \dots, n_m)$	Программа применима к входу: существует (конечный) протокол применения, то есть программа останавливается.
$\uparrow \Pi(n_1, \dots, n_m)$	Программа неприменима : работает бесконечно.
Теорема	Любая алгоритмически вычислимая частичная функция $F: \mathbb{N}^m \rightarrow \mathbb{N}$ может быть реализована на языке Милан.
Блок-схема	Графическая запись алгоритма из типовых звеньев: команда (прямоугольник), условие (ромб), начало/конец (овал).

1. Процедурная семантика

Семантика — это ответ на вопрос «что программа *делает*», тогда как грамматика из лекции 2 отвечала на вопрос «как программа *записывается*».

Процедурная семантика описывает смысл программы через **процесс выполнения**:

1. Есть программа $\Pi(x_1, \dots, x_m)$.
2. Задаём вход: $x_i \leftarrow n_i$ для $i = 1, \dots, m$.
3. Выполняем программу шаг за шагом и записываем **протокол применения** — последовательность состояний.

Возможны два исхода:

$$\downarrow \Pi(n_1, \dots, n_m) \iff \exists \text{ протокол применения } (\Pi \text{ применима к } n_1, \dots, n_m),$$

$$\uparrow \Pi(n_1, \dots, n_m) \quad (\Pi \text{ неприменима: протокол бесконечен}).$$

Пример на программе Евклида из лекции 2 [дополнение: второй случай]:

- $\downarrow \Pi(6, 3)$: протокол конечен, результат 3.
- $\uparrow \Pi(5, 0)$: условие $5 \neq 0$ всегда истинно, а $m := m - n$ оставляет $m = 5$. Цикл вечный.

Поэтому программа вычисляет **частичную** функцию: на одних входах результат есть, на других — нет. Это та же идея, что и $!f(w)$ («значение определено») из лекции 1.

2. Теорема о выразимости на Милане

Теорема. Любая алгоритмически вычислимая частичная функция $F : \mathbb{N}^m \rightarrow \mathbb{N}$ может быть реализована на языке Милан.

Что это значит простыми словами. Милан хоть и крошечный (присваивание, while, if, арифметика, ввод-вывод), но **ничего не теряет по вычислительной мощности**: всё, что вообще можно вычислить алгоритмом, можно записать и на нём.

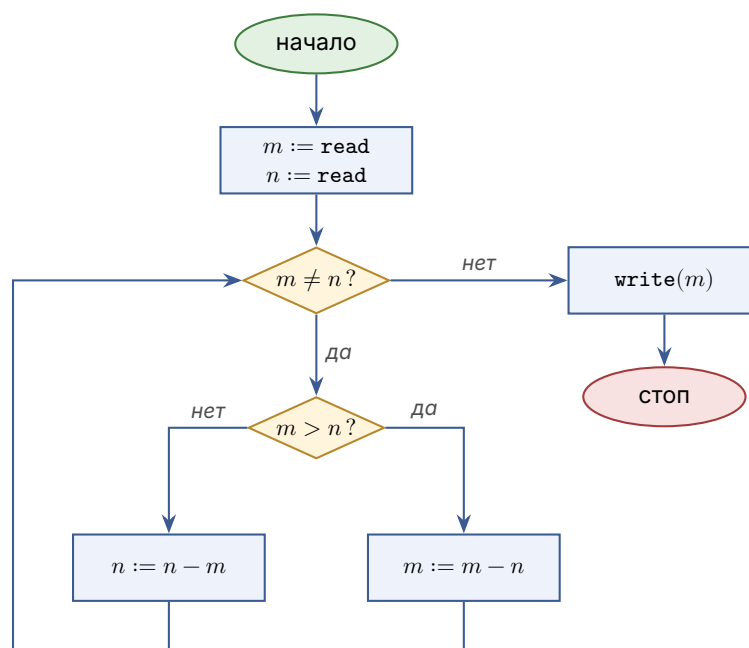
[дополнение] Такие языки называют **полными по Тьюрингу**. Ключ — наличие цикла while с произвольным условием: без него (например, только с циклом «повторить n раз») часть вычислимых функций выразить нельзя. Само понятие «алгоритмически вычислимая» формализуют через машины Тьюринга или частично рекурсивные функции; утверждение, что эти формализации охватывают всё «интуитивно вычислимое», — тезис Чёрча–Тьюринга.

3. Блок-схемы алгоритмов

Типовые звенья:

Звено	Форма	Смысл
Команда	прямоугольник, один вход и один выход	выполнить действие (например, присваивание)
Условие	ромб, один вход и два выхода: «да» / «нет»	проверить условие и пойти по одной из веток
Начало / стоп	овал	точки входа и выхода [дополнение: название формы]

Блок-схема алгоритма Евклида (перерисована из конспекта):



Блок-схема алгоритма Евклида

Сравните с текстом программы из лекции 2: ромб $m \neq n$ — это `while m <> n do ... od`, ромб $m > n$ — это `if m > n then ... else ... fi`.

4. Структурное программирование

В конспекте — список под заголовком «Структурное программирование»:

- **управляющие структуры** (циклы, условные переходы);
- **библиотеки** (процедуры, классы, функции);
- **визуальное проектирование** (IDE);
- **иерархия типов** (ООП).

[дополнение] Как это понимать. Классическое структурное программирование (Дейкстра, 1960–70-е) — идея строить программу только из трёх конструкций: **последовательность, ветвление, цикл**, без произвольных `goto`. Ровно эти конструкции есть в Милане и на блок-схеме выше. Остальные пункты списка похожи на **следующие шаги развития** технологий программирования: повторное использование кода (библиотеки), средства разработки (IDE), объектно-ориентированный подход (иерархии типов/классов). См. «Спорные места».

Спорные и неоднозначные места

1. **Символ !П** в правом верхнем углу листа. Пояснения нет. **Гипотеза:** по аналогии с $!f(w)$ из лекции 1 — «результат программы определён», то есть синоним $\downarrow \Pi$, либо обозначение значения программы. [неясно]
2. **«Протокол применения».** Определения в конспекте нет, только « $\exists$ протокол применения». **Гипотеза:** это конечная последовательность состояний выполнения, как трассировочная таблица в лекции 2.
3. **Формулировка теоремы.** В тетради исправлено «выражена» $\rightarrow$ «реализована», дописано «частичная». Взял итоговый вариант: «любая алгоритмически вычислимая *частичная* функция может быть *реализована*».
4. **Строка $x_i \leftarrow n_i$** записана под заголовком «Процедурная семантика» без пояснений. Понял её

как присваивание входных значений.

5. **Список «Структурное программирование».** Буквально под этим заголовком стоят IDE и ООП, которые к структурному программированию в узком смысле не относятся. **Гипотеза:** это перечень этапов/средств развития технологии программирования, а заголовок относится только к первому пункту, либо назван широко.
6. **Ветки на блок-схеме.** В тетради у нижнего ромба $m > n$ подписи «да»/«нет» стоят близко друг к другу. По тексту программы: «да» $\rightarrow m := m - n$, «нет» $\rightarrow n := n - m$. Так и перерисовал.

Вопросы для самопроверки

1. Чем семантика языка отличается от его синтаксиса (грамматики)?
2. Что означают записи $\downarrow \Pi(n_1, \dots, n_m)$ и $\uparrow \Pi(n_1, \dots, n_m)$?
3. Почему программа вычисляет, вообще говоря, *частичную* функцию? Приведите пример входа, на котором программа Евклида не останавливается.
4. Сформулируйте теорему о выразимости на Милане. Какая конструкция языка для неё критична?
5. Нарисуйте блок-схему программы, которая читает n и выводит $1 + 2 + \dots + n$.
6. Какие три управляющие конструкции лежат в основе структурного программирования?
7. Как изменить программу Евклида, чтобы она останавливалась на любом входе $m, n \geq 0$?
[дополнение: задача на подумать]